CT-200 Versioned Document Intelligence

Approach Document

GANJAYI SHRAVAN KUMAR | Tri9T AI Engineering Internship | 23 October 2026

1. Goal and delivered workflow

The backend turns the supplied CT-200 manuals into versioned document trees, supports browse/search/diff, preserves named source selections, and stores validated QA generation documents. Import v1, select the battery section, generate three QA ideas in offline fixture mode, import v2, then query stale outputs. The reproducible demonstration and tests exercise this sequence against the actual PDFs.

The recovered assignment brief describes behaviors rather than exact endpoint paths. Routes are therefore explicit implementation choices, documented in README and OpenAPI. No submission email has been sent.

2. Extraction and hierarchy

PyMuPDF reads native text blocks, font information, page numbers and bounding boxes. Both supplied PDFs have usable text layers, so native extraction avoids unnecessary OCR errors. Tesseract is used at 300 DPI for pages with fewer than 20 extracted characters, or when force_ocr is requested. A raster-only page fixture verifies this path; a forced-OCR run on v1 recovered the same 28 node identifiers.

Unicode NFKC normalization repairs ligatures such as fi and ff; whitespace is normalized before hashing. Original block text and geometry remain associated with each section. Bold, numbered headings identify native section boundaries, preventing numbered classification list items from becoming sections. OCR loses font metadata, so it uses numbering and short-heading heuristics and emits review warnings.

Every document has a root and numbered section nodes. Paragraphs, list blocks, table rows, figures and captions are typed content blocks owned by their section. They are not separate addressable API nodes. Children follow document order. Full text includes descendants and drives subtree hashing. The original PDFs remain available for audit and visual review.

3. Observed irregularities and handling

Repeated title: 4.2 and 7.1 are both Error Codes. Identity uses section number, never title alone.

Skipped level: 2.1.1.1 has no 2.1.1 parent. Attach it to the nearest existing prefix, 2.1, and record a warning. No synthetic source heading is invented.

Out-of-order numbering: 3.4 precedes 3.3. Preserve source order rather than numerically reordering content.

Cross-page content: paragraphs can continue after a page break, including v2 section 4.3. Keep the active section until a new heading appears. Page changes do not create nodes.

Tables: specification and error-code rows are detected using horizontally aligned lines. Preserve ordered spans and bounding boxes rather than trusting automatic table detection: initial inspection found seven fragmented table candidates on the specification page. These records support traceability, but are not a general merged-cell grid parser.

Source inconsistencies: preserve the unusual manufacturer reference in section 6.2 exactly. In v2, section 4.2 gives E3 deflation as 1.5 seconds while section 4.1 still says within 2 seconds. Record both; do not silently reconcile or alter the source.

4. Storage model and transaction boundaries

SQLAlchemy models four SQLite tables. Version holds immutable label, import sequence, original-file SHA-256 and warnings. Node holds a version-qualified ID, section key and JSON extraction record. Selection holds a unique name, pinned version and ordered unique node IDs. Generation holds its selection reference and a JSON document containing sources, prompt, provider/model, timestamp, raw attempts and validated output.

The JSON column is the document-storage layer for LLM data; a separate NoSQL server adds unnecessary setup for this small assignment. Relational metadata keeps identities and uniqueness explicit. Imports and generation saves commit atomically. Duplicate version labels and selection names return 409; missing resources return 404; invalid selections return 422. Concurrent sequence collisions return 409 for retry. SQLite and synchronous extraction suit a local demonstration, not a high-throughput multi-user service.

5. Version identity, matching and diff

Node ID is version plus section number. Cross-version matching joins section keys. Normalized subtree text has a SHA-256 hash; changed child content therefore invalidates selected parent sections too. The original PDF hash distinguishes file identity from semantic text identity. Page-number shifts alone do not trigger text staleness.

The real inputs yield 28 nodes in v1 and 29 in v2. Direct content changes occur at 2.1.1.1 (300 to 250 cycles; 15% to 10%), 3.2 (40 to 30 mmHg increments), 4.2 (E3 timing and E6 addition), and 4.3 (E1-E5 to E1-E6). Section 5.3 adds CSV export. Ancestor hashes change as expected. The diff exposes before/after text plus a token-level unified diff.

Matching failure policy: changed numbers are not assumed to be the same logical section. Renumbered sections may appear removed/added; a recycled number may be treated as a modification. Semantic matching with confidence scores and reviewer approval is a future improvement, not a claimed feature.

6. Selections, provenance and staleness

Selections are immutable and version-pinned. Duplicate names are rejected rather than silently overwritten; duplicate node IDs are removed while preserving order. Overlapping parent/child selections remain explicit, so their evidence may overlap. Generation snapshots include full source text, hashes, node IDs, section keys, version and block provenance.

Staleness is computed at retrieval against the newest import sequence or an explicit against_version. If any selected source key disappears or its subtree hash changes, the generation is stale. Filters support selection, exact version-qualified node ID and stale status. The old output and old evidence remain unchanged. A test demonstrates both changed-source invalidation and unchanged-source freshness.

7. Prompt and structured-output strategy

The live adapter uses an OpenAI-compatible chat-completions endpoint and requests JSON. The system instruction treats document content as untrusted source data. The prompt provides a Pydantic-generated schema and source IDs/text, asking for three to five QA ideas with title, test steps, expected result, exact source ID and verbatim evidence quote.

Pydantic rejects missing fields, extra fields, invalid case counts and short required text. Additional validation rejects unknown sources, quotes absent from their cited source, blank steps and duplicate case titles. Up to three attempts are allowed; failed responses trigger a repair instruction. Successful generation documents retain raw attempts for audit. Exhaustion returns 502 without saving a generation. Failed-only attempts are not persisted, a deliberate limitation of the current implementation.

Selections above 60,000 characters are rejected to bound provider requests. HTTP calls have a 45-second timeout. A provider error can be retried within the same three-attempt bound; there is no rate-limit-aware exponential backoff. Semantic grounding cannot be guaranteed by substring validation alone; an independent

entailment check and human approval are future work.

The default mock provider returns three deterministic, source-backed review templates. It makes the workflow testable without credentials but does not demonstrate model reasoning. A real provider key was unavailable; HTTP contract tests simulate the endpoint. Live quality, provider model compatibility and actual billing behavior remain unverified.

8. Validation evidence

Twelve automated tests pass. Parser tests cover missing parent levels, duplicate titles, out-of-order numbering, numbered lists, cross-page content, table rows and genuine raster OCR. API tests cover end-to-end version flow, source freshness, immutable versions, invalid input, missing nodes, duplicate selections, schema retries, retry exhaustion, false citations, missing API key and database persistence. An HTTP mock verifies authorization, endpoint path and JSON response-mode request shape.

The reproducible demo writes docs/demo-results.json with full generation provenance and actual diffs. Native extraction reports one missing-parent warning per manual. Forced v1 OCR recovered all section keys; this is not a claim of exact OCR character accuracy. A dependency warning about TestClient's HTTP transport remains, with no test failures.

9. Decision log and improvements

Native text first: preserves accurate source characters; OCR remains available and tested for scans.

Section-number identity: transparent and robust to repeated titles; limited for renumbering and document restructuring.

Subtree hashes: selections of parents detect descendant changes; layout-only changes are intentionally ignored.

SQLite plus JSON: reproducible local installation with transactional saves; a separate document database is unnecessary at this scale.

Bounded strict validation: malformed output never becomes a saved valid generation; semantic entailment still requires review.

Prioritized next work: semantic match review, full merged-cell table reconstruction, OCR confidence and page-quality scoring, figure-caption association, live provider evaluation, persistent failed-attempt audit, background ingestion jobs, authenticated access and deployment hardening.

10. Delivery status

Local source, incremental Git history, README, tests, demo evidence and this Approach Document are prepared. GitHub publication and remote repository/CI verification depend on connecting the user's GitHub account. No remote push or remote CI success is claimed in this document. Live LLM execution and general-purpose table/figure reconstruction remain the explicit incomplete areas.

References

FastAPI testing: https://fastapi.tiangolo.com/tutorial/testing/

PyMuPDF page and OCR API: https://pymupdf.readthedocs.io/en/latest/page.html

Primary assignment sources: data/ct200_manual.pdf and data/ct200_manual_v2.pdf.